📋 Cahier des charges – Système intelligent de gestion collaborative et d'optimisation de la propreté urbaine

1. Présentation générale

1.1 Contexte

La gestion de la propreté urbaine dans les villes est un défi majeur face à l'augmentation des déchets et des signalements citoyens. Le projet vise à fournir une plateforme web/mobile permettant :

- Le signalement simplifié des problèmes de propreté par les citoyens.
- Le suivi en temps réel des interventions par les agents municipaux.
- L'optimisation des tournées de nettoyage grâce à l'intelligence artificielle.

1.2 Objectifs stratégiques

- **Améliorer la réactivité** : réduire le délai entre signalement et résolution.
- **Optimiser les ressources** : diminuer les coûts opérationnels (carburant, temps) via l'IA.
- **Favoriser la collaboration citoyenne** : impliquer les habitants via des retours et un système de vote.
- **Fournir une transparence** : tableau de bord pour la municipalité et indicateurs de performance.

1.3 Périmètre

La solution inclut :

- Une interface web (responsive) et application mobile
- Un backoffice pour les gestionnaires.

- Une API REST connectée à une base de données PostgreSQL.
- Des notifications push (Firebase) pour les alertes en temps réel.
- Une cartographie interactive (Google Maps) pour visualiser les signalements et équipes.

2. Analyse des besoins

2.1 Acteurs

Acteur	Rôle
Citoyen	Signale un problème, suit l'avancement, commente, vote
Agent de terrain	Consulte les signalements assignés, met à jour le statut, utilise l'optimisation d'itinéraire
Gestionnaire / Superviseur	Gère les signalements, affecte des équipes, visualise les statistiques, valide les optimisations IA
Administrateur technique	Gère les utilisateurs, les permissions, la configuration

2.2 Cas d'utilisation principaux

- **Signalement** : le citoyen prend une photo, géolocalise, décrit le problème. Le signalement est transmis au système.
- **Affectation intelligente** : l'IA propose l'équipe la plus proche et planifie la tournée.
- **Suivi en temps réel** : l'agent voit sa feuille de route, le gestionnaire suit les interventions sur la carte.
- **Optimisation des tournées** : regroupement de signalements, calcul d'itinéraire optimal (algorithme du voyageur de commerce avec contraintes de temps).
- **Notifications** : alertes push pour nouveau signalement, changement de statut, rappel de tournée.

- **Tableau de bord** : indicateurs de performance (temps moyen de résolution, nombre de signalements, satisfaction).
- **Collaboration citoyenne** : commentaires, votes sur l'importance d'un signalement, suggestions d'amélioration.

3. Exigences fonctionnelles

3.1 Module signalement

- **F1.1** : Formulaire avec type de problème (liste prédéfinie), description, photo, géolocalisation automatique ou manuelle.
- **F1.2** : Validation des données avant envoi.
- **F1.3** : Réception d'un accusé de réception avec numéro de suivi.

3.2 Module suivi et cartographie

- **F2.1** : Affichage des signalements sur carte (marqueurs colorés selon statut).
- **F2.2** : Affichage des équipes en intervention (position temps réel si GPS).
- **F2.3** : Filtres par type, statut, zone.
- **F2.4** : Clic sur un marqueur → affichage des détails.

3.3 Module optimisation IA

- **F3.1** : Calcul d'une tournée optimale pour un agent regroupant plusieurs signalements.
- **F3.2** : Prédiction des zones à risque (historique + données temporelles).
- **F3.3** : Proposition de réaffectation d'équipes en cas d'urgence.

3.4 Module gestion des tournées

- **F4.1** : Création de tournées (manuellement ou par IA).

- **F4.2** : Visualisation des tournées du jour pour chaque agent.
- **F4.3** : Mise à jour du statut (début, en cours, terminé).

3.5 Module notification

- **F5.1** : Envoi de notification push à l'agent lorsqu'un signalement lui est assigné.
- **F5.2** : Notification au citoyen lors du changement de statut de son signalement.
- **F5.3** : Alertes de rappel pour les tournées programmées.

3.6 Module collaboration citoyenne

- **F6.1** : Commentaires sur un signalement.
- **F6.2** : Vote pour prioriser un signalement.
- **F6.3** : Forum / suggestions d'amélioration.

3.7 Module tableau de bord

- **F7.1** : Indicateurs en temps réel : nb signalements actifs, résolus, temps moyen.
- **F7.2** : Graphiques d'évolution.
- **F7.3** : Export des données (CSV/PDF).

3.8 Module administration

- **F8.1** : Gestion des utilisateurs (CRUD, rôles).
- **F8.2** : Configuration des types de problèmes, zones géographiques.
- **F8.3** : Paramétrage des notifications.

4. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Temps de chargement < 2s, API < 200ms en moyenne, support de 500 utilisateurs simultanés
Disponibilité	99,5% de disponibilité (hors maintenance programmée)
Sécurité	Authentification JWT, chiffrement des données sensibles, HTTPS, protection contre les injections SQL
Scalabilité	Architecture horizontale possible (load balancer, réplication PostgreSQL)
Maintenabilité	Code modulaire, documentation technique, tests unitaires et d'intégration
Accessibilité	Conformité RGAA (niveau A), compatible avec lecteurs d'écran
Compatibilité	Navigateurs modernes (Chrome, Firefox, Safari, Edge), responsive (mobile, tablette)

5. Architecture technique proposée

5.1 Stack

- **Frontend** : React.js (Vite) + TypeScript
- **Backend** : Node.js / Express + TypeScript
- **Base de données** : PostgreSQL 14+ avec PostGIS (pour les données géospatiales)
- **Cartographie** : Google Maps JavaScript API (ou MapLibre en open-source)
- **Notifications** : Firebase Cloud Messaging
- **Hébergement** : AWS / VPS (Docker) pour backend + base, Vercel / Netlify pour frontend
- **Gestion de version** : Git (GitHub/GitLab)

5.2 Diagramme d'architecture

```
[Citoyen] → (HTTPS) → [CDN/Reverse Proxy] → [React App]
                                ↓
                          [API Gateway]
                                ↓
                      [Node.js Backend]
                      /        |        \
               [PostgreSQL] [Firebase] [Google Maps]
```

5.3 Modèle de données simplifié

- **users** (id, email, password, role)
- **reports** (id, user_id, type, description, photo_url, status, assigned_agent_id, created_at, resolved_at)
- **tours** (id, agent_id, planned_start, planned_end, route_points (JSON), status)
- **tour_reports** (tour_id, report_id) – relation many-to-many
- **comments** (id, report_id, user_id, content, created_at)
- **votes** (id, report_id, user_id, value)
- **optimization_logs** (id, generated_at, recommendations (JSON), applied)

6. Interface utilisateur (maquettes)

6.1 Écrans principaux (web)

- Page d'accueil / Tableau de bord
- Carte interactive avec filtres
- Liste des signalements avec options de tri
- Formulaire de signalement (avec upload photo et géolocalisation)
- Espace agent : tournée du jour, itinéraire optimisé
- Espace gestionnaire : statistiques, affectation, configuration
- Profil et paramètres

6.2 Mobile

- Adaptation responsive (menu en bas)
- Utilisation native de l'appareil photo et GPS
- Notifications push

6.3 Charte graphique

- Couleurs principales : vert (#2b7a4b), blanc, gris clair
- Icônes : Font Awesome

7. Contraintes techniques

- **Clé Google Maps** : à fournir par le maître d'ouvrage.
- **Firebase** : compte Firebase avec projet configuré.
- **Hébergement** : budget mensuel estimé pour serveur (environ 50-100 €/mois en phase pilote) or Netlify.
- **Conformité RGPD** : gestion des données personnelles (photos, localisation) avec consentement.

8. Planning prévisionnel (6 mois)

Phase	Durée	Livrables
Conception	3 semaines	Maquettes, modélisation DB, spécifications techniques
Développement backend	6 semaines	API, modèles, tests unitaires
Développement frontend	8 semaines	Interfaces web et mobile, intégration Google Maps
Intégration IA / optimisation	4 semaines	Algorithme de clustering, itinéraire optimal
Tests & recette	3 semaines	Plan de test, correction bugs

Phase	Durée	Livrables
Déploiement & formation	2 semaines	Documentation utilisateur, formation des agents

9. Risques et mesures d'atténuation

Risque	Probabilité	Impact	Atténuation
Changement de périmètre	Moyen	Élevé	Valider les spécifications en phase de conception, méthodologie agile
Performance de l'algorithme IA	Faible	Moyen	Optimisation progressive, caching, scaling
Dépendance Google Maps	Moyen	Élevé	Prévoir une solution de repli (OpenStreetMap)
Adoption par les agents	Moyen	Moyen	Formation, interface simple, valeur ajoutée visible (gain de temps)

10. Budget estimatif (indicatif)

- **Développement** : 4–6 mois (2 développeurs full-stack, 1 designer UI/UX) – selon les taux en vigueur.
- **Hébergement** : 100–200 €/mois (serveur + base + CDN)
- **Licences** : Google Maps (environ 200-400 €/mois selon utilisation), Firebase (gratuit jusqu'à un certain seuil)
- **Maintenance** : 15-20% du coût de développement par an

✅ Conclusion

Ce cahier des charges constitue une base solide pour le lancement du projet. Il définit les besoins fonctionnels, techniques et organisationnels. Une fois validé, il servira de référence pour la réalisation de la plateforme.

Rédigé par : Najib Abdelilah, Sanae Belhaj, Amine Boussaboune

Date : 27 mars 2026 – **Version** : 1.0